Section 2: Time and Space Complexity

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: LeetCode DSA Foundations for Interviews

Section 2: Time and Space Complexity

- SLIDE 11: Section Intro, Time and Space Complexity
- SLIDE 12: What Big-O Actually Measures
- SLIDE 13: Complexity Classes and Practical Input Sizes
- SLIDE 14: Analyzing Time Complexity: Loops and Nested Work
- SLIDE 15: Recursion, Space Complexity, and Amortized Cost
- SLIDE 16: How Interviewers Assess Efficiency
- SLIDE 17: Worked Example: Three Solutions to the Same Problem
- SLIDE 18: Wrong vs Right: Complexity Analysis Mistakes
- SLIDE 19: Interview Practice: Complexity Analysis
- SLIDE 20: Section Cheat Sheet

C11: LeetCode DSA Foundations for Interviews

Section 2: Time and Space Complexity

SLIDE 11: Section Intro, Time and Space Complexity

Section 2: Time and Space Complexity

Every solution you write in a coding interview will be evaluated on two dimensions: how fast it runs (time complexity) and how much memory it uses (space complexity). Stating these correctly and unprompted is one of the clearest signals of a prepared candidate. Stating them incorrectly is worse than not stating them at all, because it tells the interviewer you think you understand something you do not.

This section gives you the tools to analyze any piece of code and state its complexity with confidence.

What We'll Cover:

1. What Big-O notation actually measures and why constants are dropped
2. Every complexity class from O(1) to O(n!) with practical input size limits
3. How to analyze loops: single, nested, consecutive, and halving
4. Recursion complexity, space complexity, and amortized analysis
5. How interviewers use complexity questions to assess your depth
6. Worked example: three solutions to the same problem at different complexity levels

Where This Fits in the Course:

In Section 1 we introduced the five-step framework. Step 5 was “state complexity unprompted.” This section teaches you how to do that correctly. Every section that follows, from arrays to backtracking, will reference the complexity analysis techniques you learn here. When we say “the brute force is O(n^2) and the optimal is O(n)” in later sections, you will understand exactly what that means and how to verify it yourself.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 12: What Big-O Actually Measures

Growth Rate, Not Runtime

Big-O describes how the number of operations grows as the input size grows. It does not tell you the actual runtime in seconds. An O(n) algorithm is not “fast” and an O(n^2) algorithm is not “slow” in absolute terms. At n=10, an O(n^2) algorithm performs 100 operations, which completes in microseconds on any modern machine.

Big-O matters when n gets large. At n=100,000, the difference between O(n) and O(n^2) is the difference between 100,000 operations and 10,000,000,000 operations. That is the difference between finishing in milliseconds and timing out.

Why Constants and Lower-Order Terms Are Dropped

Consider the function f(n) = 3n^2 + 5n + 100. As n grows:

n	3n^2	5n	100	Total	n^2 as % of Total
10	300	50	100	450	67%
100	30,000	500	100	30,600	98%
1,000	3,000,000	5,000	100	3,005,100	99.8%
10,000	300,000,000	50,000	100	300,050,100	99.98%

At n=10,000, the n^2 term accounts for 99.98% of the total. The 5n and the constant 100 are irrelevant. This is why 3n^2 + 5n + 100 simplifies to O(n^2). The constant multiplier 3 is dropped because Big-O describes the shape of the growth curve, not the exact count.

Concrete Comparison: O(n) vs O(n^2)

```
Algorithm A: O(n)     → processes 100,000 elements in ~1 ms
Algorithm B: O(n^2)   → processes 100,000 elements in ~10,000,000 ms (~2.7 hours)

At n = 100,000:
  A does      100,000 operations
  B does 10,000,000,000 operations

B does 100,000x more work than A.
At n = 10, B only does 10x more work. The gap widens as n grows.
```

Common Misconception:

“O(1) means instant.” No. O(1) means the operation count does not change with input size. A HashMap lookup is O(1) but might take longer in absolute time than scanning a 3-element array, because of hashing overhead. Big-O is about scaling behavior, not speed at small inputs.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 13: Complexity Classes and Practical Input Sizes

The Complexity Landscape

Every complexity class has a practical ceiling: the largest input size that can be processed within a typical 1-2 second time limit in an interview or competitive programming judge. Knowing these limits lets you reverse-engineer the expected

approach from the problem constraints.

Class	Name	Operations at n=10	Operations at n=1,000	Operations at n=10^6	Max n (1-2 sec)	Example Algorithm	Interview Acceptability
O(1)	Constant	1	1	1	Any	HashMap lookup, array access	Always acceptable
O(log n)	Logarithmic	3	10	20	Any	Binary search	Always acceptable
O(n)	Linear	10	1,000	10^6	~10^8	Single loop, linear scan	Almost alway optimal
O(n log n)	Linearithmic	33	10,000	2 × 10^7	~10^6 - 10^7	Merge sort, Arrays.sort()	Standard for sort-based solutions
O(n^2)	Quadratic	100	10^6	10^12	~10^4	Nested loops, brute force pair checking	Acceptable fo n <= 10,000
O(n^3)	Cubic	1,000	10^9	10^18	~500-1,000	Triple nested loops	Rarely acceptable
O(2^n)	Exponential	1,024	10^301	Infinite	~20-25	Subsets, naive recursion	Only for very small n
O(n!)	Factorial	3,628,800	Infinite	Infinite	~10-12	Permutations	Only for tiny inputs

The Critical Boundary: O(n) to O(n^2)

This is the most important practical boundary in interview problem solving. Most brute force solutions are O(n^2) because they use nested loops. Most optimal solutions are O(n) or O(n log n) because they replace the inner loop with a hash map lookup, two-pointer technique, or sorting step.

When the problem constraint says n <= 10^5 or n <= 10^6, the interviewer is telling you: your solution must be O(n) or O(n log n). An O(n^2) approach will not pass. This is the constraint-to-complexity mapping from Section 1 (Slide 5) in action.

Using Constraints to Identify the Expected Approach:

```
Read the constraint on n:

n <= 10       → O(n!) is fine    → brute force, backtracking
n <= 20-25    → O(2^n) is fine   → bitmask, subset enumeration
n <= 10^4     → O(n^2) is fine   → nested loops acceptable
n <= 10^5-6   → O(n log n) max   → sort + scan, or O(n) with hash map
n >= 10^7     → O(n) or better   → single pass, two pointers, math
```

Interview Tip: Before writing any code, look at the constraint on n. It narrows your options significantly. If n can be 10^5, you know nested loops will time out and you need a hash map, sorting, or two-pointer approach.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 14: Analyzing Time Complexity: Loops and Nested Work

The Four Loop Patterns

Almost every iterative algorithm is built from four loop patterns. Once you recognize which pattern is present, stating the complexity is mechanical.

Pattern 1: Single Loop, O(n)

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

The loop body runs n times. Each iteration does O(1) work (one addition, one array access). Total: O(n).

Pattern 2: Nested Loops, Multiply, O(n^2)

```
int count = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        count++;
    }
}
```

The outer loop runs n times. For each outer iteration, the inner loop runs n times. Total: n x n = O(n^2). Three levels of nesting would be O(n^3).

Pattern 3: Consecutive Loops, Add, O(n)

```
for (int i = 0; i < n; i++) {
    processA(arr[i]);
}
for (int j = 0; j < n; j++) {
    processB(arr[j]);
}
```

First loop: O(n). Second loop: O(n). Total: O(n) + O(n) = O(2n) = O(n). Consecutive loops add; the dominant term wins. Five consecutive O(n) loops is still O(n).

Pattern 4: Halving Loop, O(log n)

```
int i = n;
while (i > 0) {
    i = i / 2;
}
```

The value halves each iteration: n, n/2, n/4, ..., 1. Number of iterations: log2(n). This is the pattern behind binary search. A loop that doubles instead of halving (i *= 2 until i >= n) is also O(log n).

The Rule for Combined Patterns:

Nested loops: multiply their individual complexities.
Consecutive loops: add, then take the dominant term.

Pattern	Structure	Complexity	Example
Single loop	`for i in 0..n`	O(n)	Linear scan
Nested loops	`for i` { `for j` }	O(n^2)	Brute force pair check
Consecutive loops	`for i` then `for j`	O(n) + O(n) = O(n)	Two-pass algorithms
Halving loop	`while i > 0: i /= 2`	O(log n)	Binary search
Loop inside halving	`while` { `for j in 0..n` }	O(n log n)	Merge sort merge step
Nested with different bounds	`for i in 0..n` { `for j in 0..m` }	O(n * m)	Matrix traversal

Hidden Costs: The Body of the Loop Matters

The loop structure alone does not determine complexity. What happens inside the loop body matters.

```
for (int i = 0; i < n; i++) {
    if (list.contains(arr[i])) {
        // ...
    }
}
```

This looks like a single O(n) loop. But `ArrayList.contains()` is O(n) because it scans the entire list. So the total is O(n) x O(n) = O(n^2).

Hidden cost operations in Java:

Operation	Time Complexity	Why
`ArrayList.contains(x)`	O(n)	Linear scan of the list
`ArrayList.remove(0)`	O(n)	Shifts all elements left
`String += otherString` (in loop)	O(n) per concat	Creates new String object each time
`String.indexOf(substring)`	O(n * m)	Naive substring search
`HashSet.contains(x)`	O(1) average	Hash-based lookup
`HashMap.get(key)`	O(1) average	Hash-based lookup

Interview Tip: When analyzing complexity, do not just count loop levels. Examine every operation inside the loop body. A single-loop solution that calls `.contains()` on an ArrayList inside the loop is O(n^2), not O(n). This is one of the most common analysis mistakes freshers make.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 15: Recursion, Space Complexity, and Amortized Cost

Recursion Complexity

Recursion creates a tree of calls. The total work is the number of nodes in that tree multiplied by the work per node.

The key question: how many branches does each call create, and how does the input size reduce?

Recursion Pattern	Branches	Input Reduction	Total Calls	Complexity	Example
Linear	1	Reduce by 1	n	O(n)	Factorial, linked list traversal
Halving	1	Divide by 2	log n	O(log n)	Binary search (recursive)
Divide and conquer	2	Divide by 2	n (across all levels)	O(n log n)	Merge sort
Binary branching	2	Reduce by 1	2^n	O(2^n)	Naive Fibonacci, subsets
Full branching	n	Reduce by 1	n!	O(n!)	Permutations

Worked Inline: Naive Fibonacci

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Each call makes 2 recursive calls, input reduces by 1. This creates a binary tree of depth n. Total nodes: approximately 2^n. Time complexity: O(2^n).

```
fib(5) calls fib(4) and fib(3)
fib(4) calls fib(3) and fib(2)
```

```
fib(3) calls fib(2) and fib(1)
...

fib(5) computes fib(3) twice, fib(2) three times.
At n=50, this takes over a minute. At n=100, it won't finish in your lifetime.
```

Adding memoization (storing computed results) drops this to O(n) because each subproblem is solved once. This is the foundation of dynamic programming, covered in C6.

Space Complexity

Space complexity measures the additional memory your algorithm uses, not counting the input itself. This is called auxiliary space.

Space Usage	Complexity	Example
Fixed number of variables	O(1)	Two pointers, Kadane's algorithm
Array or HashMap of size n	O(n)	Hash map for Two Sum, frequency array
2D matrix n x m	O(n * m)	Grid DP table
Recursion stack of depth d	O(d)	Recursive DFS with depth d
Recursion with n-sized structure per level	O(n * d)	Building new strings at each recursive call

The Distinction: Input Space vs Auxiliary Space

When an interviewer asks "what is the space complexity?", they typically mean auxiliary space. If you are given an array of n elements and you only use three integer variables, the space complexity is O(1), not O(n). The input already exists and is not counted against your algorithm.

If the problem says "solve this in-place", it means O(1) auxiliary space. You may modify the input array but you may not allocate a new one.

Amortized Analysis

Some operations are usually O(1) but occasionally O(n). If the expensive operation happens rarely enough, the average cost per operation is still O(1). This is amortized O(1).

```
ArrayList.add():
  - Usually O(1): append to the end of the internal array
  - Occasionally O(n): when the internal array is full, allocate a new array
    of double the size and copy all elements

  Over n insertions: n cheap appends + log(n) expensive copies
  Total copy work: 1 + 2 + 4 + 8 + ... + n = ~2n
  Average per insertion: 2n / n = O(1) amortized
```

For interview purposes, know that `ArrayList.add()` and `HashMap.put()` are O(1) amortized. If asked to explain why, describe the doubling and copying as shown above.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 16: How Interviewers Assess Efficiency

The Journey from Brute Force to Optimal

Most interview problems are structured so that the brute force solution is O(n^2) and the optimal solution is O(n) or O(n log n). The interview is designed around the journey between these two. The interviewer is evaluating whether you can identify why the brute force is slow and what technique eliminates the unnecessary work.

What You Should Do:

1. State the brute force approach and its complexity

2. Identify specifically what makes it slow (usually: “the inner loop scans the entire array to find a match”)
3. Propose the improvement and name the technique
4. Implement the optimal solution
5. State the final time AND space complexity unprompted

Interviewer Follow-Up Signals

When an interviewer asks a follow-up question, they are often hinting at a specific complexity class. Learning to decode these hints saves time and shows awareness.

What the Interviewer Says	What They Mean	Target Complexity
“Can you do better?”	Drop one complexity class	One level lower (e.g., O(n^2) to O(n log n))
“Can you do this in one pass?”	O(n) exists	O(n) with hash map or running state
“What if the array is sorted?”	Leverage sorted order	O(log n) binary search or O(n) two pointers
“What if memory is not a concern?”	Trade space for time	Use hash map/set for O(1) lookups
“Can you do this in-place?”	O(1) auxiliary space	Two pointers, swap-based techniques
“What is the time complexity of your solution?”	Verify you can analyze your own code	State it correctly with justification
“Can you improve the space?”	Reduce from O(n) to O(1)	Remove the auxiliary data structure

The Complexity Staircase

```
Interview progression for a typical problem:

Brute Force:    O(n^2) time, O(1) space    "Nested loops check every pair"
Sort + Scan:    O(n log n) time, O(1) space "Sort first, then two pointers"
Hash Map:       O(n) time, O(n) space       "Replace inner loop with lookup"

The interviewer evaluates:
  - Can you identify the brute force? (baseline competency)
  - Can you explain WHY it's slow? (analytical ability)
  - Can you propose the improvement? (pattern recognition)
  - Can you implement it correctly? (coding ability)
  - Can you state complexity unprompted? (professional maturity)
```

Interview Tip: Always state both time AND space complexity without being asked. Saying “This runs in O(n) time and O(n) space because of the HashMap” is significantly stronger than waiting for the interviewer to ask. It demonstrates that you evaluate trade-offs, not just correctness.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 17: Worked Example: Three Solutions to the Same Problem

Problem: Two Sum (LeetCode 1)

Given an integer array `nums` and an integer `target`, return the indices of the two numbers that add up to `target`. You may assume each input has exactly one solution, and you may not use the same element twice.

This problem is used here because it clearly demonstrates three distinct complexity trade-offs. The full Two Sum pattern is covered in depth in Section 5 (Hash Maps).

Solution 1: Brute Force with Nested Loops

Check every pair of elements. For each element, scan the rest of the array for the complement.

```java
public int[] twoSum(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
```

```
            if (nums[i] + nums[j] == target) {
                return new int[]{i, j};
            }
        }
    }
    return new int[]{};
}
```

Analysis:

- Time: O(n^2). Outer loop runs n times, inner loop runs up to n-1 times per outer iteration.
- Space: O(1). Only a few integer variables regardless of input size.
- Trade-off: No extra memory used, but slow at scale. At n = 100,000, this performs up to 5 billion comparisons.

Solution 2: Sort Then Two Pointers

Sort the array, then use two pointers from opposite ends. If sum is too small, move left pointer right. If too large, move right pointer left.

```
public int[] twoSum(int[] nums, int target) {
    int[][] indexed = new int[nums.length][2];
    for (int i = 0; i < nums.length; i++) {
        indexed[i] = new int[]{nums[i], i};
    }
    Arrays.sort(indexed, (a, b) -> Integer.compare(a[0], b[0]));

    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = indexed[left][0] + indexed[right][0];
        if (sum == target) {
            return new int[]{indexed[left][1], indexed[right][1]};
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{};
}
```

Analysis:

- Time: O(n log n). Sorting dominates. The two-pointer scan after sorting is O(n).
- Space: O(n). The indexed array stores original indices because sorting rearranges elements.
- Trade-off: Faster than brute force but requires extra space to preserve indices. If the problem asked for values instead of indices, this could be O(1) space.

Solution 3: Hash Map in One Pass

For each element, check if its complement (target - current) already exists in the map. If yes, return both indices. If no, store the current element.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
```

```
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

Analysis:

- Time: O(n). Single pass through the array. Each HashMap operation is O(1) amortized.
- Space: O(n). The HashMap stores up to n entries.
- Trade-off: Fastest approach but uses O(n) additional memory.

Trade-Off Summary:

Solution	Time	Space	Trade-Off
Nested Loops	O(n^2)	O(1)	No extra memory, but too slow for large n
Sort + Two Pointers	O(n log n)	O(n)	Faster, but sorting cost and index tracking
Hash Map	O(n)	O(n)	Fastest, trades memory for speed

Key Takeaway: The progression from O(n^2) to O(n) is not magic. Each improvement has a specific mechanism: Solution 2 eliminates random scanning by sorting, Solution 3 eliminates scanning entirely by using O(1) hash lookups. Understanding the mechanism is more important than memorizing the solutions.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 18: Wrong vs Right: Complexity Analysis Mistakes

Mistake 1: Ignoring the Loop Body

```
// What a fresher writes:
for (int i = 0; i < n; i++) {
    if (list.contains(arr[i])) {
        result.add(arr[i]);
    }
}
```

Wrong: "O(n) because there is one for loop."

Right: "O(n^2) because the for loop runs n times, and `ArrayList.contains()` inside the loop is O(n), performing a linear scan for each iteration. The total is n x n = O(n^2)."

Fix: Replace ArrayList with HashSet. `HashSet.contains()` is O(1) average, making the total O(n).

Mistake 2: Confusing Nested and Consecutive Loops

```
// Two consecutive loops, NOT nested:
for (int i = 0; i < n; i++) {
    processA(arr[i]);
}
for (int j = 0; j < n; j++) {
    processB(arr[j]);
}
```

Wrong: “O(n^2) because there are two loops.”

Right: “O(n) because the loops are consecutive, not nested. We add: O(n) + O(n) = O(2n) = O(n).”

The test: does the second loop run inside the first, or after the first? Nested loops multiply. Consecutive loops add.

Mistake 3: Forgetting Space Complexity

Wrong: “My solution is O(n) using a HashMap.”

Right: “My solution is O(n) time and O(n) space. The HashMap stores up to n key-value pairs. If space is a concern, two pointers on sorted input gives O(n log n) time with O(1) space.”

Interviewers expect you to state both. Omitting space complexity suggests you are not thinking about trade-offs.

Mistake 4: Missing Hidden Costs in Java

```
// String concatenation in a loop:
String result = "";
for (int i = 0; i < n; i++) {
    result += arr[i];
}
```

Wrong: “O(n) because there is one loop.”

Right: “O(n^2) because String is immutable in Java. Each `+=` creates a new String object, copying all previous characters. The total copy work is 1 + 2 + 3 + ... + n = n(n+1)/2 = O(n^2).”

Fix: Use `StringBuilder`, which gives O(n) total via amortized O(1) appends.

Mistake 5: Claiming O(n) for a Sort-Based Solution

Wrong: “I sort the array, then do a single scan. The scan is O(n), so the solution is O(n).”

Right: “I sort the array in O(n log n), then do a single scan in O(n). The total is O(n log n) + O(n) = O(n log n). The sort dominates.”

Always include the preprocessing cost in the total complexity.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 19: Interview Practice: Complexity Analysis

Practice Prompt

Analyze the time and space complexity of the following code and explain your reasoning:

```
public static List<Integer> findDuplicates(int[] arr) {
    List<Integer> result = new ArrayList<>();
    Set<Integer> seen = new HashSet<>();
    for (int i = 0; i < arr.length; i++) {
        if (!seen.add(arr[i])) {
            result.add(arr[i]);
        }
    }
    return result;
}
```

Expected Walkthrough:

Step 1: Identify the loop structure. There is a single for loop running n times, where n is `arr.length`.

Step 2: Analyze the loop body. `seen.add()` is a HashSet operation, O(1) amortized. `result.add()` is an ArrayList append, O(1) amortized. Both are O(1) per iteration.

Step 3: State time complexity. "The loop runs n times, each iteration does O(1) work, so the time complexity is O(n)."

Step 4: State space complexity. "The HashSet `seen` stores up to n elements. The result list stores up to n elements in the worst case. The auxiliary space is O(n)."

Step 5: State it cleanly. "Time O(n), space O(n). The HashSet trades O(n) memory for O(1) per-element duplicate detection, compared to nested loops which would be O(n^2) time but O(1) space."

Now Analyze This One:

```
public static boolean hasDuplicateInRange(int[] arr, int k) {
    for (int i = 0; i < arr.length; i++) {
        for (int j = i + 1; j <= i + k && j < arr.length; j++) {
            if (arr[i] == arr[j]) return true;
        }
    }
    return false;
}
```

This is trickier. The inner loop runs at most k iterations per outer iteration, not n. Total operations: n x k. If k is a constant (like k=5), this is O(n). If k can scale with n (like k = n/2), this is O(n^2). The answer depends on the relationship between k and n. In an interview, you should state: "The time complexity is O(n * k). If k is constant, this simplifies to O(n). If k can be proportional to n, this is O(n^2)." Distinguishing these cases shows the interviewer you understand what variables matter.

Common Interviewer Follow-Ups:

Follow-Up	Expected Response
"Is that worst case or average case?"	"O(n) is both average and worst case here because the loop runs n times regardless of input values."
"What if I said you can't use extra space?"	"Without a HashSet, I would sort the array first O(n log n) and check adjacent elements in O(n), or use nested loops at O(n^2)."
"What is the best case?"	"O(1) if the first two elements are duplicates, but we analyze worst case for Big-O."

Common Fresher Mistakes:

- Saying "O(n)" for any code with one visible loop, without checking what happens inside the loop body
- Saying "O(n^2)" for any code with two loops, without checking whether they are nested or consecutive
- Not mentioning space complexity at all, forcing the interviewer to ask for it
- Confusing amortized O(1) with worst case O(1). When the interviewer asks "is HashMap.get always O(1)?", the correct answer is "O(1) amortized, O(n) worst case due to hash collisions, but in practice with a good hash function, we treat it as O(1)"

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 20: Section Cheat Sheet

Complexity Class Quick Reference

Class	Name	Max n (1-2 sec)	Common In
O(1)	Constant	Any	Hash lookups, array access, math formulas
O(log n)	Logarithmic	Any	Binary search, balanced BST operations

Class	Name	Max n (1-2 sec)	Common In
O(n)	Linear	~10^8	Single pass, hash map solutions, two pointers
O(n log n)	Linearithmic	~10^6-7	Sorting, divide and conquer
O(n^2)	Quadratic	~10^4	Nested loops, brute force
O(2^n)	Exponential	~20-25	Subsets, backtracking on small inputs
O(n!)	Factorial	~10-12	Permutations

Analysis Rules

Structure	Rule	Result
Single loop (0 to n)	Count iterations	O(n)
Nested loops	Multiply	O(n^2), O(n^3), ...
Consecutive loops	Add, take dominant	O(n) + O(n) = O(n)
Loop that halves input	Count halvings	O(log n)
Loop inside halving loop	Multiply	O(n log n)
Recursion: count tree nodes	Branches^depth	O(2^n), O(n!), etc.
Space: auxiliary structures	Size of structures	O(n), O(n^2), etc.
Space: recursion stack	Depth of recursion	O(n), O(log n), etc.

Input Size to Expected Complexity

```
n <= 10       → O(n!) acceptable   → backtracking, permutations
n <= 20-25    → O(2^n) acceptable  → bitmask, subset enumeration
n <= 500      → O(n^3) acceptable  → triple nested loops
n <= 10^4     → O(n^2) acceptable  → nested loops, brute force
n <= 10^5-6   → O(n log n) needed  → sort + scan, or O(n) hash map
n >= 10^7     → O(n) or O(log n)   → single pass, binary search, math
```

Common Hidden-Cost Operations in Java

Operation	Actual Cost	Safe Alternative
`String += other` in loop	O(n) per concat, O(n^2) total	`StringBuilder.append()` O(1) amortized
`ArrayList.contains(x)`	O(n) linear scan	`HashSet.contains(x)` O(1) amortized
`ArrayList.remove(0)`	O(n) shift all elements	`ArrayDeque.pollFirst()` O(1)
`Collections.sort()`	O(n log n) time, O(n) space	Know space cost exists (TimSort)
`Arrays.sort(int[])`	O(n log n) time, O(log n) space	Dual-pivot quicksort, not stable
`String.substring()`	O(j-i) creates new String	Acceptable, but not free

How This Connects to Other Sections

- Section 3 (Arrays): operation cost awareness from this section determines whether your array solution is actually O(n) or secretly O(n^2)
- Section 5 (Hash Maps): understanding O(1) amortized lookup is why hash maps reduce O(n^2) brute forces to O(n)
- Section 6 (Two Pointers): understanding O(n) total pointer movement across iterations is key to analyzing two-pointer solutions
- Section 7 (Sliding Window): the amortized argument for the inner while loop being O(n) total is the same pattern as amortized ArrayList.add()
- Section 8 (Binary Search): O(log n) from the halving loop pattern is the foundation of binary search analysis
- Section 9 (Sorting): knowing that comparison sorts cannot beat O(n log n) worst case sets the baseline for sort-based solutions

- Section 12 (Recursion): recursion complexity analysis rules from this section are applied directly when analyzing backtracking solutions

Common Interview Mistakes

Mistake	Why It's Wrong	Better Approach
"O(n) because one for loop"	Ignores cost of operations inside the loop body	"O(n^2) because the loop runs n times and .contains() inside is O(n)"
"O(n^2) because two loops"	Confuses consecutive with nested	"O(n) because the loops are consecutive, not nested. We add, not multiply."
Stating only time complexity	Ignores space trade-off	"O(n) time, O(n) space due to the HashMap"
"O(n log n)" without mentioning sort	Complexity seems to appear from nowhere	"I sort in O(n log n) then scan in O(n), so total is O(n log n)"
"O(1) amortized" without being able to explain	Shows memorization without understanding	"Each resize doubles capacity and copies n elements, but this happens every n insertions, so amortized cost is O(1) per insert"

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina